

正负样本不均衡问题解决方案报告

一句话总结：正负样本不均衡问题的解决方案可以从三个层面理解：**Loss 层面**通过 Focal Loss 按样本难度动态加权、Dice Loss 用区域重叠比天然归一化、GHM 按梯度分布抑制易样本和噪声，从数学机制上让少数类获得足够的梯度信号；**架构层面**通过 CenterNet 等 Anchor-Free 范式将正负比从 1:73000 降至 1:640，配合 SE/CBAM 等动态通道注意力让网络自动聚焦少数类特征，以及 ATSS/SimOTA/TOOD 等动态标签分配策略自适应确定正样本阈值，从网络设计上减少不均衡的根源；**训练策略层面**通过类别均衡采样、Mixup/Mosaic 增强少数类多样性、梯度累积保证每次更新包含足够的少数类信息。实际项目中这些方法通常组合使用，其中 Focal Loss + Dice Loss 处理像素级不均衡、Anchor-Free 或动态分配处理候选级不均衡、注意力机制处理特征级不均衡，三者互补是最常见且有效的组合方案。

目录

- 1. 问题概述
- 2. 网络设计层面
- 3. 训练策略层面
- 4. 部署与推理层面
- 5. 各解决方案具体实现指南
- 6. 各方法对比总结
- 7. 实践建议

1. 问题概述

1.1 什么是正负样本不均衡

在分类或检测任务中，正负样本数量存在显著差异。例如：

- 欺诈检测：**正常交易 99.9%，欺诈交易 0.1%
- 医学影像：**健康样本远多于病灶样本
- 目标检测：**背景区域（负样本）远多于前景目标（正样本）

1.2 不平衡带来的核心问题

问题	说明
模型偏向多数类	损失函数被多数类主导，少数类被忽略
准确率幻觉	99:1 的数据集全预测为负也能达到 99% 准确率
梯度淹没	大量简单负样本产生的小梯度淹没了少量正样本的有效梯度
泛化能力差	对少数类（通常也是关注类）的召回率极低

1.3 不平衡程度的度量

- **Imbalance Ratio (IR)**: 多数类 / 少数类的样本比
- **Skewness**: 数据分布的偏度
- 一般 $IR > 10$ 即需特殊处理, $IR > 100$ 属于极端不平衡

2. 网络设计层面

2.1 损失函数重设计

2.1.1 Focal Loss

核心思想: 降低容易分类样本的损失权重，让模型聚焦于难分类样本。

公式:

$$FL(p_t) = -\alpha_t \times (1 - p_t)^\gamma \times \log(p_t)$$

- p_t : 模型对真实类别的预测概率
- γ (focusing parameter): 调节对难样本的聚焦程度，通常取 2.0
- α (balancing factor): 正负样本权重，通常正样本取 0.25

原理详解: 在标准交叉熵 $-\log(p_t)$ 前面乘了一个 $(1 - p_t)^\gamma$ 调制因子。大量易分类样本原本会贡献大量但无信息量的累积损失，Focal Loss 通过幂函数的指数压制让这些样本"闭嘴"，使梯度更新主要由难样本驱动。调制因子 $(1 - p_t)^\gamma$ 的曲线特性类似于图像处理中的 gamma 校正——压缩一端、保留另一端，本质是同一个 $y = x^\gamma$ 幂函数变换。

对容易样本 (p_t 接近 1)：

- $p_t = 0.9, \gamma = 2 \rightarrow (1 - 0.9)^2 = 0.01$ ，损失压缩到原来的 **1%**
- $p_t = 0.99, \gamma = 2 \rightarrow (1 - 0.99)^2 = 0.0001$ ，几乎为零

对困难样本 (p_t 接近 0.5 或更低)：

- $p_t = 0.5, \gamma = 2 \rightarrow (1 - 0.5)^2 = 0.25$ ，损失只压缩到 25%
- $p_t = 0.1, \gamma = 2 \rightarrow (1 - 0.1)^2 = 0.81$ ，损失几乎不变

样本类型	p_t	调制因子 $(1 - p_t)^\gamma$	效果
极易样本	0.99	0.0001	损失接近0，几乎不贡献梯度
易样本	0.9	0.01	损失大幅衰减
中等样本	0.5	0.25	损失适度衰减
难样本	0.1	0.81	损失基本保留

与标准交叉熵、加权交叉熵的对比：

	CE	Weighted CE	Focal Loss
类别权重	无	固定 α	固定 α
样本难度感知	无	无	有 $(1 - p_t)^\gamma$
易样本损失	完整保留	等比缩放	指数压制
难样本损失	完整保留	等比缩放	基本保留

数值例子 ($\gamma = 2, \alpha = 1$)：

样本	p_t	CE	Focal Loss	比值
易样本	0.9	0.105	0.001	1%
中等	0.5	0.693	0.173	25%
难样本	0.1	2.302	1.865	81%

CE 对所有样本"一视同仁"，加权 CE 对类别"区别对待但一成不变"，Focal Loss 对每个样本实时评估难度并动态调整权重——这就是关键区别。

参数选择建议：

参数	推荐值	说明
$\gamma = 0$	退化为 CE Loss	无聚焦效果
$\gamma = 1$	温和聚焦	适中度不均衡
$\gamma = 2$	标准设置	RetinaNet 原始设定，IR 10~1000
$\gamma = 5$	强聚焦	极端不均衡场景
α	0.25 ~ 0.75	根据正负比例调整

优势：不需要额外的采样策略，端到端训练
出处：Lin et al., "Focal Loss for Dense Object Detection" (ICCV 2017)

2.1.2 类别加权交叉熵 (Weighted Cross Entropy)

公式：

$$WCE = -w_{pos} \times y \times \log(p) - w_{neg} \times (1-y) \times \log(1-p)$$

权重设置策略：

- **逆频率法：** $w_i = N / (C \times n_i)$ ，其中 N 为总样本数，C 为类别数， n_i 为第 i 类样本数
- **中位数频率法：** $w_i = \text{median}(\text{freq}) / \text{freq}_i$
- **有效样本数法：** $w_i = 1 / E_n(n_i)$ ，其中 $E_n = (1-\beta^n)/(1-\beta)$

2.1.3 GHM (Gradient Harmonized Mechanism)

核心思想：从梯度分布角度出发，抑制大量简单样本和极少数异常难样本的影响。GHM 不直接修改 loss 函数本身，而是按**梯度分布的统计信息**动态调整每个样本的权重，是一个通用的梯度重加权框架。

公式：

$$L_{GHM} = w_i \times L_{CE}(x_i)$$

其中权重 $w_i = \frac{1}{GD(g_i)}$ ， g_i 是样本 i 的梯度范数， $GD(g)$ 是梯度值 g 处的样本密度。

梯度范数的含义与计算：

对于交叉熵损失，梯度范数恰好等于 $1 - p_t$ ：

$\partial \text{CE} / \partial x = p_t - 1 \rightarrow g = |p_t - 1| = 1 - p_t$

不需要额外计算反向传播，就是模型预测概率的补数。

p_t	梯度范数 $g = 1 - p_t$	含义
0.99	0.01	模型很确定，梯度小
0.7	0.3	有点不确定
0.5	0.5	完全没把握
0.1	0.9	预测很离谱，梯度很大

GHM 的计算过程：

- 1. 算出所有样本的 $g_i = 1 - p_{t,i}$
- 2. 把 $[0, 1]$ 区间分成若干个桶（如 100 个）
- 3. 统计每个桶里的样本数，除以桶宽 $\rightarrow$ 得到密度 $GD(g)$
- 4. 权重 $w = \frac{1}{GD(g)}$

梯度大小	含义	密度	GHM 权重
小 ($p_t \rightarrow 1$)	易样本	极多	很低（被压制）
中等	有价值的难样本	适中	适中
大 ($p_t \rightarrow 0$)	极难样本/离群噪声	少但有害	也低（也被压制）

相比 Focal Loss 的关键改进：

Focal Loss 的问题：极难样本不会被压制。 $(1 - 0.01)^2 = 0.98$ ，离群噪声样本的 loss 几乎完整保留，可能污染训练。

GHM 的优势：

- 易样本多 $\rightarrow$ 密度高 $\rightarrow$ 权重低 $\checkmark$
- 极难样本（离群噪声）虽然梯度大，但数量相对也形成小高峰 $\rightarrow$ 权重也被压低 $\checkmark$
- 只有"中等难度、数量适中"的样本获得最高权重

代价是 GHM 需要统计梯度分布，计算开销比 Focal Loss 稍大。

本质：GHM 是"梯度层面的采样均衡器"，不是新的 loss 函数，而是加在已有 loss 上的动态加权策略。

优势：同时解决了简单负样本过多和离群点噪声的问题

出处：Li et al., "Gradient Harmonized Single-stage Detector" (AAAI 2019)

2.1.4 Dice Loss / Generalized Dice Loss

适用场景：语义分割中的类别不均衡

公式：

$$\text{Dice} = 2|X \cap Y| / (|X| + |Y|) = 2TP / (2TP + FP + FN)$$

Dice 系数是精确率和召回率的**调和平均**， $\text{Dice Loss} = 1 - \text{Dice}$ 。

为什么能解决类别不均衡：

假设一张 100×100 的图，前景只占 1%（100 像素），背景占 99%（9900 像素）。

- **CE 的问题：**背景 9900 个像素的 loss 求和，完全压倒前景 100 个像素。即使前景全预测错了，总 loss 也变化不大——模型感受不到前景的错误。
- **Dice 的优势：**这是一个比值，分母 $2TP + FP + FN$ 自动根据前景规模归一化，不管前景多小，漏检就使 Dice 直接趋向 0，背景预测再好也无法补偿。

场景	TP	FP	FN	Dice
前景只检出 50/100	50	0	50	0.67
前景全部漏检	0	0	100	0
背景全部预测对但前景漏了	0	0	100	0

调和平均的数学本质：调和平均的特质是只要其中一个（精确率或召回率）很低，整体就被拉低。算术平均允许一个高一个低来补偿，调和平均不允许。

与 CE 的对比：

	CE	Dice Loss
度量方式	逐像素求和取平均	区域重叠比
对类别大小的敏感度	大类主导梯度	自动归一化，大小类等权
漏检小目标	损失变化微小	Dice 直接暴跌
优化目标	逐像素正确率	结构级重叠度

Focal Loss 与 Dice Loss 的对比：

	Focal Loss	Dice Loss
作用层面	逐像素	整体区域
解决思路	按难度加权，让难样本说话	按重叠度评分，让小目标不被忽视
抗不均衡机制	压制多数易样本的梯度	比值归一化，大小类天然等权
本质	CE 的改良	全新的区域匹配度量

Focal Loss 是**微观**视角——调整每个像素的贡献权重。Dice Loss 是**宏观**视角——直接评估预测区域和真实区域的重合程度。两者经常组合使用： $L = L_{Focal} + \lambda \cdot L_{Dice}$ 。

特点：直接优化类别重叠度，天然对类别比例不敏感

2.1.5 OHEM (Online Hard Example Mining)

核心思想：在训练过程中在线挖掘难样本，只用损失最大的样本参与梯度更新。

流程：

- 1. 前向传播计算所有样本损失
- 2. 按损失排序
- 3. 选取 top-K（或 top-P%）的难样本
- 4. 仅用这些样本计算梯度回传

实现变体：

- **标准 OHEM：**直接取 loss 最大的样本
- **带正样本保护 OHEM：**确保正样本不被过滤掉
- **分区域 OHEM（目标检测）：**对不同 IoU 区间分别做难例挖掘

2.1.6 损失函数选择速查表

不均衡程度	任务类型	推荐损失函数
轻度 (IR < 10)	分类	Weighted CE
中度 (IR 10~100)	分类	Focal Loss
中度 (IR 10~100)	目标检测	Focal Loss + OHEM

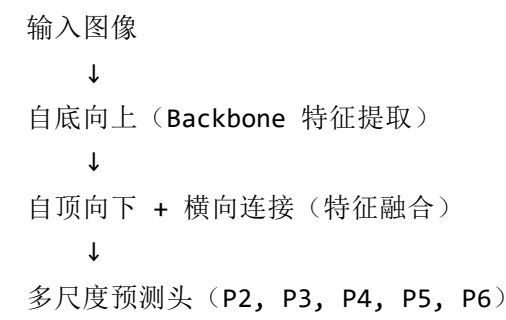
不平衡程度	任务类型	推荐损失函数
重度 (IR > 100)	分类	Focal Loss + 采样
重度 (IR > 100)	分割	Generalized Dice Loss
重度 (IR > 100)	目标检测	Focal Loss + GHM

2.2 网络架构改进

2.2.1 特征金字塔网络 (FPN)

解决思路： 目标检测中，不同尺度的目标本身存在不平衡（小目标远少于大目标背景）。

关键设计：

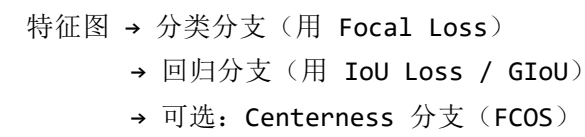


缓解不平衡的机制：

- 小目标在高分辨率特征图上获得更多正样本分配
- 每层独立做分类和回归，避免跨尺度竞争

2.2.2 解耦检测头 (Decoupled Head)

思路： 将分类和回归分支分离，各自使用独立的损失权重。



好处： 分类分支可以独立调节正负样本比例，不受定位分支干扰。

2.2.3 CenterNet / Anchor-Free 范式

思路：摒弃 Anchor，用中心点 heatmap 表示目标，大幅减少正负样本不均衡。核心在于语义转变——从"粗筛框"变成"精确定位点"，候选数量自然减少一个量级。

不均衡对比（以 640×640 输入为例）：

	候选数量	正样本数（10个目标）	正负比
Anchor-Based（9 anchors/位置）	~368 万	~50	1:73000
CenterNet（heatmap 80×80）	6400	10	1:640

正负比从 1:73000 降到 1:640，缩小约 100 倍。配合 Focal Loss 进一步压制剩余负样本，不需要复杂的正负样本采样策略。

对比：

方面	Anchor-Based	Anchor-Free (CenterNet)
正负定义	IoU 阈值匹配	中心点落在哪个格子
正样本数	每个目标可能匹配多个 Anchor	每个目标仅对应一个网格点
正负比	约 1:1000	约 1:100（更均衡）
不均衡程度	严重	相对温和

2.2.4 注意力机制

适用方向：让网络更关注少数类特征。

SE-Block、CBAM、ECA-Net 都是**输入依赖的动态注意力**——不同输入图片会产生不同的通道权重。注意区分两个层次的"权重"：

层次	含义	性质
参数权重	FC/Conv 的 W, b	初始化后通过训练学习
注意力权重	最终输出的 $\alpha_1, \alpha_2, \dots$	每张图不同，动态计算

各模块流程：

SE-Block：

输入 X ($C \times H \times W$)
→ GAP → (C 维向量)
→ $FC(C \rightarrow C/r) \rightarrow ReLU \rightarrow FC(C/r \rightarrow C) \rightarrow Sigmoid$
→ 输出 α (C 维注意力权重)
→ $\alpha \odot X$

CBAM 通道注意力：

输入 X
→ GAP 和 GMaxPool 并行 → 两个 (C 维向量)
→ 共享 MLP → 相加 → Sigmoid
→ 输出 α

ECA-Net：

输入 X
→ GAP → (C 维向量)
→ $1D\ Conv(kernel=k) \rightarrow Sigmoid$
→ 输出 α

各组件职责：

组件	职责
GAP	压缩信息，提取每个通道的统计摘要（每个通道在空间维度取平均）
FC + ReLU + FC	真正学习通道间组合关系 ，非线性来自 ReLU
Sigmoid	把结果压到 [0,1]，方便做加权乘法（不是提供非线性的主要来源）
梯度回传	学习信号，驱动 MLP 学会正确的映射

梯度回传机制：

前向过程中，GAP 把每个通道压缩成"该通道的平均激活强度"，MLP 根据所有通道的激活模式组合判断哪些通道对当前任务重要，Sigmoid 输出 [0,1] 范围的缩放系数。

反向传播时，因为 $\hat{X} = \alpha \odot X$ ：

$$\partial L / \partial \alpha_c = \sum_{\{i,j\}} (\partial L / \partial \hat{X}_c(i,j)) \times X_c(i,j)$$

含义：通道 c 上所有空间位置的梯度乘以该通道的原始值求和 → 告诉模型该通道的注意力权重应调大还是调小。梯度继续通过标准全连接层反向传播更新 W_1, W_2 。

参数初始化：FC/Conv 层用 Kaiming 初始化，偏置初始化为 0，通过正常反向传播训练。本质上是学习一个映射 $f: z \rightarrow \alpha$ ，从"各通道激活统计"映射到"各通道重要性评分"。

常用模块：

模块	原理	适用场景
SE-Block	通道注意力，重标定特征通道权重	常规不均衡
CBAM	通道 + 空间注意力	小目标 / 少数类位置不固定
ECA-Net	高效通道注意力	轻量化需求
Transformer Attention	全局自注意力	大模型，需要全局上下文

2.3 标签分配策略改进

2.3.1 动态标签分配

传统方法使用固定 IoU 阈值（如 0.5）划分正负样本，存在以下问题：

- 阈值太高：正样本过少
- 阈值太低：引入低质量正样本

现代动态分配策略：

方法	核心思路	特点
ATSS (CVPR 2020)	每层自适应计算 IoU 阈值	根据目标统计特性动态调整
SimOTA (YOLOX)	最优传输分配， 代价矩阵决定正样本	自动平衡正负， 无需手动调阈值
OTA (CVPR 2021)	将标签分配建模为最优传输问题	理论最优
TaskAlignedAssigner (TOOD)	分类 + 对齐联合度量	分类和检测质量对齐

ATSS (Adaptive Training Sample Selection) 详细过程

核心思想：根据统计特征自适应确定 IoU 阈值。

对每个 GT box:

1. 在每个 FPN 层找距离 GT 中心最近的 $k=9$ 个 anchor
2. 计算这 k 个 anchor 与 GT 的 IoU
3. 计算这 k 个 IoU 的均值 μ 和标准差 σ
4. 阈值 $T = \mu + \sigma$
5. $\text{IoU} > T$ 且中心在 GT 内的 anchor $\rightarrow$ 正样本

特点：阈值不是人工设定的，而是根据当前 anchor-GT 的匹配质量自适应计算的。不同 GT、不同 FPN 层、不同训练阶段，阈值都不同。

OTA (Optimal Transport Assignment) 详细过程

核心思想：把正负样本分配建模为最优传输问题。

将标签分配看作运输问题：

- GT = 供应商，提供若干个"正样本名额"
- Anchor = 需求方，需要被分配为正样本或负样本
- 运输成本 = 分类损失 + 回归损失 (cost 越低越好)

用 Sinkhorn-Knopp 算法求解最优分配方案

特点：全局最优视角，一个 anchor 被分配给哪个 GT 是在全局层面优化的，不是局部贪心。

SimOTA (Simplified OTA) 详细过程

核心思想：OTA 的简化版，去掉 Sinkhorn 迭代。

对每个 GT:

1. 计算所有 anchor 与该 GT 的 $\text{cost} = \text{cls_loss} + \text{reg_loss}$
2. 选 cost 最小的 top-k 个 anchor 作为候选正样本
3. k 的值动态确定：取 GT 与所有 anchor 的 IoU 之和再取整

特点：保留了 OTA 的"按 cost 匹配"思想，但用简单的 top-k 替代复杂的求解器，速度快得多。YOLOX 用的就是这个。

TOOD (Task-aligned One-stage Object Detection) 详细过程

核心思想：让分类和定位两个任务的监督信号对齐。

定义对齐度量：

$$t = s^{\alpha} \cdot u^{\beta}$$

s = 分类预测分数

u = 预测框与 GT 的 IoU

α, β = 可调超参数

正样本分配：

对每个 GT，按 t 值选 top-k 的 anchor 为正样本

Loss 设计：

分类 **target** 不再是 $\{0,1\}$ ，而是 t 值本身

→ 分类分支被引导去预测"分类+定位的综合质量"

特点：之前的分配方法只看 IoU 或只看 cost，TOOD 认为分类好但定位差的 anchor 不该是高质量正样本，所以用 $s^{\alpha} \cdot u^{\beta}$ 综合评估。

四种方法对比

	正样本选择依据	全局/局部	计算复杂度
ATSS	IoU 统计阈值	每个 GT 独立	低
OTA	最优传输 cost	全局最优	高（Sinkhorn）
SimOTA	top-k cost	每个 GT 独立	中
TOOD	分类×定位对齐度	每个 GT 独立	低

演进脉络：

ATSS: IoU 统计自适应阈值（告别人工阈值）

↓

OTA: 全局最优分配（引入最优传输理论）

↓

SimOTA: 简化版全局分配（工程实用的妥协）

↓

TOOD: 任务对齐（不只关注"谁是正样本"，还关注"正样本的质量评分"）

3. 训练策略层面

3.1 数据采样策略

3.1.1 过采样 (Oversampling)

随机过采样 (ROS):

对少数类随机复制样本，直到与多数类数量相当

- 优点：简单直接
- 缺点：容易过拟合少数类
- 适用：数据量较小的场景

SMOTE (Synthetic Minority Oversampling):

1. 对少数类每个样本 x_i ，找到其 k 个最近邻
2. 随机选择一个近邻 x_j
3. 在 x_i 和 x_j 之间随机插值生成新样本：
$$x_{\text{new}} = x_i + \lambda \times (x_j - x_i), \lambda \in (0, 1)$$

变体:

- **Borderline-SMOTE**: 仅在决策边界附近生成样本
- **ADASYN**: 根据样本难度自适应调整生成数量
- **SMOTE-ENN**: 生成后用 ENN 清洗重叠样本

深度学习增强版:

方法	说明
GAN 过采样	用 GAN 生成少数类新样本
VAE 过采样	用 VAE 在潜在空间插值生成
Mixup / CutMix	混合不同类样本，隐式增加少数类特征

3.1.2 欠采样 (Undersampling)

随机欠采样 (RUS):

随机丢弃多数类样本，使正负比例均衡

- 优点：训练加速
- 缺点：信息损失
- 适用：多数类数据量极大，且存在大量冗余

聚类欠采样:

1. 对多数类做 K-Means 聚类
2. 从每个聚类中心附近保留少量样本
3. 保留代表性，减少冗余

Tomek Links:

找到最近邻但标签不同的样本对，删除多数类样本
→ 清理决策边界，不是严格意义的欠采样

3.1.3 混合采样

推荐做法:

对少数类做轻度过采样（如 SMOTE）
+ 对多数类做轻度欠采样（如聚类采样）
→ 最终比例控制在 1:3 到 1:5（不必完全 1:1）

3.1.4 采样策略选择指南

- 数据量 < 1万：SMOTE / 过采样 + 数据增强
- 数据量 1万~100万：混合采样
- 数据量 > 100万：欠采样 + OHEM / Focal Loss（无需额外采样）

3.2 数据增强

3.2.1 少数类专用增强

仅对少数类应用更强的数据增强：

```
if label == minority_class:
    apply: RandomCrop + ColorJitter + Rotate + Flip + ...
else:
    apply: RandomFlip only
```

3.2.2 高级增强策略

方法	原理	适用场景
Mixup	两张图线性混合： $x = \lambda x_1 + (1-\lambda)x_2$	通用分类
CutMix	将一张图的矩形区域粘贴到另一张	检测 / 分类
Mosaic	4 张图拼接为 1 张（YOLO 系列标配）	目标检测
Copy-Paste	将少数类目标复制粘贴到不同背景	实例分割
Class-Balanced Augmentation	按类别反比例调整增强强度	通用

3.2.3 Copy-Paste 增强流程（目标检测/分割）

- 从源图像中提取少数类实例（使用已有标注）
- 对实例做随机变换（缩放、旋转、翻转）
- 粘贴到目标图像的随机位置
- 更新目标图像的标注
- 可选：添加边缘模糊使粘贴更自然

3.2.4 实现方式总览

数据增强在 PyTorch 中的实现分三类：**torchvision 直接提供**、**第三方库提供**、**需要自己写**。

torchvision.transforms 直接可用的：

增强方法	接口	说明
随机翻转	RandomHorizontalFlip / RandomVerticalFlip	最常用
随机裁剪	RandomCrop / RandomResizedCrop	
颜色抖动	ColorJitter(brightness, contrast, saturation)	
随机旋转	RandomRotation	
高斯模糊	GaussianBlur	
随机仿射	RandomAffine	旋转+平移+缩放组合
Normalize	Normalize(mean, std)	标配

第三方库提供的（推荐 timm 和 ultralytics）：

增强方法	库	说明
Mixup + CutMix	timm.data.mixup.Mixup	一行调用，支持混合概率控制
Mosaic	ultralytics	YOLO 系列标配， 配置文件加一行即可
Copy-Paste	ultralytics	同上
RandAugment	torchvision.transforms.RandAugment	自动搜索增强策略组合

需要自己写但很简单：

增强方法	难度	说明
类别差异化增强	低	加个 if 判断标签，对不同类用不同 transform
红外专用增强	中	如自适应直方图均衡、噪声注入等
自定义 Copy-Paste	中	需要操作标注信息

3.2.5 数据增强在代码中的位置

数据增强在 Dataset 的 __getitem__ 中执行，DataLoader 负责批次加载时自动调用。每个样本在 __getitem__ 被取出时随机增强一次，所以同一个样本在不同 epoch 拿到的增强结果不同。

```

from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

# 训练集 transform (带增强)
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(0.5),
    transforms.RandomVerticalFlip(0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.Normalize(mean=[...], std=[...]),
])

# 验证集 transform (不做随机增强)
val_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.Normalize(mean=[...], std=[...]),
])

```

```

class MyDataset(Dataset):
    def __init__(self, images, labels, transform=None):
        self.images = images
        self.labels = labels
        self.transform = transform

    def __getitem__(self, idx):
        img = self.images[idx]
        label = self.labels[idx]
        if self.transform:
            img = self.transform(img)    # 在这里做增强
        return img, label

```

```

# 训练集用增强，验证集不用
train_dataset = MyDataset(train_imgs, train_labels, transform=train_transform)
val_dataset = MyDataset(val_imgs, val_labels, transform=val_transform)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)

```

特殊处理：Mixup / CutMix 是 batch 级别的增强，不在 Dataset 里，而是在训练循环中对整个 batch 操作：

```
from timm.data.mixup import Mixup

mixup = Mixup(mixup_alpha=0.8, cutmix_alpha=1.0, prob=1.0)

for images, labels in train_loader:
    images, labels = mixup(images, labels) # batch级增强在训练循环里
    loss = criterion(model(images), labels)
    loss.backward()
    optimizer.step()
```

红外小目标检测常用增强组合：

```
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(0.5),
    transforms.RandomVerticalFlip(0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.Normalize(mean=[...], std=[...]),
])
```

如果使用 YOLO 系列框架（如 ultralytics），Mosaic 和 Copy-Paste 只需在配置文件里设置：

```
mosaic: 1.0 # 开启 Mosaic
copy_paste: 0.1 # Copy-Paste 概率
```

3.3 训练流程优化

3.3.1 两阶段训练

阶段一：用均衡数据（采样后）训练模型至收敛

→ 让模型学到少数类的基本特征

阶段二：用原始不均衡数据微调

→ 让模型适应真实分布

→ 学习率降低 10 倍

→ 可配合 Focal Loss

3.3.2 课程学习 (Curriculum Learning)

- Epoch 1~20: 使用均衡子集 (1:1 采样)
- Epoch 21~50: 逐步过渡到真实分布 (1:1 → 1:5 → 1:10 → 真实比例)
- Epoch 51~100: 使用真实分布 + Focal Loss

3.3.3 元学习 / Few-shot 思路

当少数类样本极少时 (< 10 个):

- 1. 用多数类数据预训练特征提取器
- 2. 冻结 backbone, 只训练分类头
- 3. 使用 Prototypical Network 的思路:
 - 计算每个类的原型 (类中心)
 - 分类时计算到各原型的距离

3.4 评估指标选择

3.4.1 不均衡场景下应避免使用的指标

- 准确率 (Accuracy): 在不均衡下无意义
- 全局 Loss: 被多数类主导

3.4.2 推荐指标

指标	公式/说明	关注点
Precision	$TP / (TP + FP)$	预测为正的准确度
Recall	$TP / (TP + FN)$	正样本被找到的比例
F1-Score	$2 \times P \times R / (P + R)$	P 和 R 的调和平均
PR-AUC	PR 曲线下面积	对不均衡更鲁棒
ROC-AUC	ROC 曲线下面积	阈值无关的整体评估
mAP@0.5	检测任务	各类 AP 的平均
mAP@0.5:0.95	检测任务	更严格的评估

3.4.3 混淆矩阵分析

必须按类别单独分析：

- 少数类的 Recall 是否达标？
- 少数类的 Precision 是否可接受？
- 是否存在系统性的假阴性？

3.5 学习率与优化器调优

3.5.1 分层学习率

对少数类相关层使用更大的学习率

```
param_groups = [  
    {'params': backbone.parameters(), 'lr': 1e-4},          # backbone 小学习率  
    {'params': detection_head.parameters(), 'lr': 1e-3},    # 检测头大学习率  
    {'params': minority_class_layers.parameters(), 'lr': 5e-3}, # 少数类相关最大  
]
```

3.5.2 梯度累积

当 batch 中少数类样本过少时：

1. 使用小 batch + 多次前向传播
2. 累积梯度后再做一次反向传播
3. 确保每次参数更新包含足够的少数类信息

```
accumulation_steps = 4  
for i, batch in enumerate(dataloader):  
    loss = model(batch) / accumulation_steps  
    loss.backward()  
    if (i + 1) % accumulation_steps == 0:  
        optimizer.step()  
        optimizer.zero_grad()
```

3.6 Batch 构造策略

3.6.1 Class-Balanced Batch Sampling

```
# 每个 batch 保证一定的正样本比例
# 而非完全随机采样

batch_size = 32
pos_per_batch = 8 # 强制每个 batch 至少 8 个正样本
neg_per_batch = 24

pos_indices = random.sample(pos_pool, pos_per_batch)
neg_indices = random.sample(neg_pool, neg_per_batch)
batch = construct_batch(pos_indices + neg_indices)
```

3.6.2 N-Pair Sampling

每个 batch 包含：

- 1 个 anchor（少数类样本）
- 1 个 positive（同类不同样本）
- N-1 个 negative（多数类样本）

→ 适合度量学习 / 对比学习框架

4. 部署与推理层面

4.1 阈值优化

4.1.1 默认阈值的问题

默认阈值 0.5 在不均衡场景下几乎总是次优的：

- 若正:负 = 1:100，阈值 0.5 会导致召回率极低
- 需要根据业务需求调整阈值

4.1.2 阈值搜索方法

方法一：Youden's J 统计量

$$J = \text{Sensitivity} + \text{Specificity} - 1$$

选择使 J 最大化的阈值

方法二：F1-Score 最优化

在验证集上遍历阈值 $[0.01, 0.99]$:

$\text{best_thresh} = \text{argmax}_t F1(t)$

方法三：业务约束驱动

场景：欺诈检测，要求召回率 $\geq 95\%$

做法：从高到低降低阈值，直到 Recall 达到 95%

代价：Precision 会下降（更多误报）

方法四：代价敏感阈值

定义代价矩阵：

	预测正	预测负
实际正	0	C_{fn} （漏检代价）
实际负	C_{fp} （误报代价）	0

最优阈值： $t^* = C_{fn} / (C_{fn} + C_{fp})$

4.1.3 类别级独立阈值

不同类别使用不同阈值：

- 少数类：降低阈值（如 0.3）→ 提高召回率
- 多数类：提高阈值（如 0.7）→ 减少误报

通过验证集上每个类别单独搜索最优阈值

4.2 模型校准 (Calibration)

4.2.1 为什么需要校准

不均衡训练后，模型的输出概率往往不准确：

- Focal Loss / 过采样训练的模型，输出概率偏离真实概率
- 需要校准使 $P(\text{预测正确}) \approx \text{模型输出置信度}$

4.2.2 校准方法

方法	原理	适用场景
Temperature Scaling	学习一个温度系数 T，将 logits 除以 T 后再 softmax	最简单有效，推荐首选
Platt Scaling	在输出 logits 上拟合一个逻辑回归	二分类
Isotonic Regression	非参数的保序回归	概率分布不规则时
Histogram Binning	将概率分桶，用实际频率替换	数据充足时

Temperature Scaling 实现：

```
# 在验证集上学习最优温度 T
import torch
import torch.nn as nn

class TemperatureScaling(nn.Module):
    def __init__(self):
        super().__init__()
        self.temperature = nn.Parameter(torch.ones(1))

    def forward(self, logits):
        return logits / self.temperature

# 用 NLL Loss 优化 T（只优化 T，不优化模型）
# 通常 T 在 1.0 ~ 3.0 之间
```


4.3 后处理策略

4.3.1 规则辅助过滤

模型输出 + 业务规则联合决策：

```
if model_score > 0.3 AND meets_business_rules:
    predict_positive()
elif model_score > 0.7:
    predict_positive() # 高置信度直接判正
else:
    predict_negative()
```

4.3.2 级联策略 (Cascading)

级联一（高召回）：宽松阈值 + 轻量模型

- 召回率 > 99%，允许大量误报
- 过滤掉明确的负样本

级联二（高精度）：严格模型 + 正常阈值

- 对级联一的候选做精确判断
- 提升最终 Precision

效果：整体 F1 显著提升，推理开销增加有限

4.3.3 后处理 NMS 优化（目标检测）

标准 NMS：按置信度排序，去除重叠框

- 问题：少数类（小目标）容易被误删

Soft-NMS：

- 不直接删除，而是降低重叠框的分数
- $\text{decay_score} = \text{score} \times (1 - \text{IoU})$

Class-Aware NMS：

- 每个类别独立做 NMS
- 避免少数类被多数类的高分框抑制

4.4 在线学习与模型更新

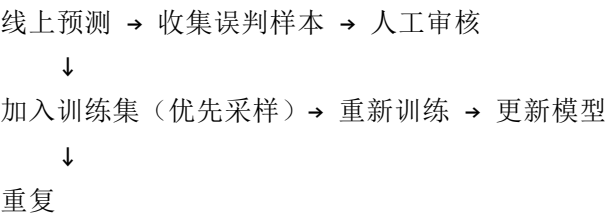
4.4.1 增量学习

部署后持续收集少数类样本：

- 1. 记录模型低置信度的预测
- 2. 人工标注这些样本
- 3. 定期（如每周）用新样本微调模型
- 4. 使用较小的学习率和较少的 epoch
- 5. 配合 EWC / L2 正则防止遗忘

4.4.2 难例挖掘 Pipeline

生产环境反馈循环：



4.5 模型集成

4.5.1 不均衡场景下的集成策略

策略	说明
不同采样 + 相同模型	分别用过采样、欠采样、原始数据训练 3 个模型，投票/平均
相同数据 + 不同损失	分别用 CE、Focal Loss、Dice Loss 训练，融合
不同阈值 + 相同模型	同一模型取 3 个不同阈值的预测，加权融合
Bagging + 欠采样	EasyEnsemble / BalanceCascade

4.5.2 EasyEnsemble 流程

1. 将多数类划分为 T 个子集，每个子集大小 = 少数类大小
2. 每个子集 + 全部少数类 $\rightarrow$ 训练一个基学习器
3. T 个基学习器投票 / 平均

多数类 (10000) $\rightarrow$ 分为 10 个子集 (每个 1000)

少数类 (1000) $\rightarrow$ 每次都全部使用

$\rightarrow$ 训练 10 个模型 $\rightarrow$ 集成

5. 各解决方案具体实现指南

本章节说明报告中提到的每个解决方案在 PyTorch 中如何实现：是直接调用接口还是需要手写、代码放在哪里、有哪些现成的框架支持。

5.1 损失函数实现

5.1.1 Focal Loss

实现方式：需要自己写（PyTorch 没有内置），但代码很短。

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2.0, reduction='mean'):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, logits, targets):
        ce_loss = F.binary_cross_entropy_with_logits(logits, targets, reduction='none')
        p_t = torch.exp(-ce_loss) # p_t = sigmoid(logit) 如果预测正确
        alpha_t = self.alpha * targets + (1 - self.alpha) * (1 - targets)
        focal_weight = alpha_t * (1 - p_t) ** self.gamma
        loss = focal_weight * ce_loss

        if self.reduction == 'mean':
            return loss.mean()
        elif self.reduction == 'sum':
            return loss.sum()
        return loss

```

使用方式：替换标准交叉熵即可，不需要改网络结构。

```

criterion = FocalLoss(alpha=0.25, gamma=2.0)
loss = criterion(model_output, target)

```

5.1.2 加权交叉熵

实现方式： torch.nn.CrossEntropyLoss 直接支持 weight 参数。

```
import torch.nn as nn

# 计算类别权重（逆频率法）
class_counts = [10000, 100] # 各类样本数
weights = [sum(class_counts) / (len(class_counts) * c) for c in class_counts]
weights = torch.tensor(weights, dtype=torch.float32)

criterion = nn.CrossEntropyLoss(weight=weights.to(device))
loss = criterion(output, target)
```

5.1.3 Dice Loss

实现方式：需要自己写，代码简短。

```
class DiceLoss(nn.Module):
    def __init__(self, smooth=1.0):
        super().__init__()
        self.smooth = smooth

    def forward(self, preds, targets):
        preds = torch.sigmoid(preds)
        intersection = (preds * targets).sum(dim=(2, 3))
        union = preds.sum(dim=(2, 3)) + targets.sum(dim=(2, 3))
        dice = (2.0 * intersection + self.smooth) / (union + self.smooth)
        return 1.0 - dice.mean()
```

组合使用（红外小目标检测常见做法）：

```
focal = FocalLoss(alpha=0.25, gamma=2.0)
dice = DiceLoss(smooth=1.0)
loss = focal(logits, targets) + 1.0 * dice(logits, targets)
```

5.1.4 GHM

实现方式：需要自己写，比 Focal Loss 稍复杂，需要统计梯度分布。

```

class GHMC(nn.Module):
    def __init__(self, bins=100, momentum=0.0):
        super().__init__()
        self.bins = bins
        self.momentum = momentum
        self.register_buffer('edges', torch.linspace(0, 1, bins + 1))
        self.register_buffer('acc_sum', torch.zeros(bins))

    def forward(self, logits, targets):
        # g = 1 - p_t, 即梯度范数
        probs = torch.sigmoid(logits)
        p_t = targets * probs + (1 - targets) * (1 - probs)
        g = 1.0 - p_t

        # 统计梯度密度
        weights = torch.zeros_like(g)
        for i in range(self.bins):
            mask = (g >= self.edges[i]) & (g < self.edges[i + 1])
            count = mask.sum().float()
            if count > 0:
                density = count / self.bins
                weights[mask] = 1.0 / density

        ce_loss = F.binary_cross_entropy_with_logits(logits, targets, reduction='none')
        return (weights * ce_loss).mean()

```

5.2 网络架构实现

5.2.1 SE-Block 注意力

实现方式：需要自己写模块，然后插入到现有网络中。timm 库中有很多内置实现。

```

class SEBlock(nn.Module):
    def __init__(self, channels, reduction=16):
        super().__init__()
        mid = channels // reduction
        self.squeeze = nn.AdaptiveAvgPool2d(1)      # GAP
        self.excitation = nn.Sequential(
            nn.Linear(channels, mid),               # FC 降维
            nn.ReLU(inplace=True),                  # 非线性
            nn.Linear(mid, channels),               # FC 升维
            nn.Sigmoid()                            # 归一化到 [0,1]
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        s = self.squeeze(x).view(b, c)              # (B, C)
        s = self.excitation(s).view(b, c, 1, 1)     # (B, C, 1, 1)
        return x * s                                # 逐通道缩放

```

如何插入到现有网络：

在 ResNet 的残差块中加入 SE

```

class SEBasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.se = SEBlock(out_channels, reduction=16) # 插入 SE 模块

    def forward(self, x):
        identity = x
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = self.se(out)                          # 在残差相加前应用 SE
        out += identity
        return F.relu(out)

```

5.2.2 ECA-Net（更轻量的通道注意力）

```
class ECABlock(nn.Module):
    def __init__(self, channels, gamma=2, b=1):
        super().__init__()
        # 自适应计算 1D 卷积核大小
        kernel_size = int(abs((math.log2(channels) + b) / gamma))
        kernel_size = kernel_size if kernel_size % 2 else kernel_size + 1

        self.squeeze = nn.AdaptiveAvgPool2d(1)
        self.conv = nn.Conv1d(1, 1, kernel_size, padding=kernel_size // 2, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        b, c, _, _ = x.size()
        s = self.squeeze(x).view(b, 1, c)                # (B, 1, C)
        s = self.conv(s)                                  # 1D 卷积替代 FC
        s = self.sigmoid(s).view(b, c, 1, 1)
        return x * s
```


5.2.3 CBAM (通道 + 空间双重注意力)

```
class ChannelAttention(nn.Module):
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)
        self.mlp = nn.Sequential(
            nn.Linear(channels, channels // reduction),
            nn.ReLU(),
            nn.Linear(channels // reduction, channels),
        )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        b, c, _, _ = x.size()
        avg_out = self.mlp(self.avg_pool(x).view(b, c))
        max_out = self.mlp(self.max_pool(x).view(b, c))
        s = self.sigmoid(avg_out + max_out).view(b, c, 1, 1)
        return x * s

class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super().__init__()
        self.conv = nn.Conv2d(2, 1, kernel_size, padding=kernel_size // 2, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = x.mean(dim=1, keepdim=True)
        max_out = x.max(dim=1, keepdim=True)[0]
        s = torch.cat([avg_out, max_out], dim=1)
        s = self.sigmoid(self.conv(s))
        return x * s

class CBAM(nn.Module):
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.ca = ChannelAttention(channels, reduction)
        self.sa = SpatialAttention()

    def forward(self, x):
        x = self.ca(x)          # 先通道注意力
```

```
x = self.sa(x)          # 再空间注意力
return x
```

5.3 动态标签分配实现

5.3.1 框架支持情况

方法	框架支持	说明
ATSS	mmdetection 内置	mmdet/models/dense_heads/atss_head.py
OTA	mmdetection 内置	mmdet/models/dense_heads/ota
SimOTA	mmdetection / ultralytics	YOLOX 和 YOLOv8 默认使用
TOOD	mmdetection 内置	mmdet/models/dense_heads/tood_head.py

5.3.2 使用 mmdetection 的 ATSS

```
# 在配置文件中指定
model = dict(
    type='ATSS',
    backbone=...,
    neck=...,
    bbox_head=dict(
        type='ATSSHead',
        num_classes=1,                # 红外小目标：单类
        anchor_generator=dict(
            type='AnchorGenerator',
            strides=[8, 16, 32, 64, 128],
            scales=[8],
            ratios=[1.0],              # 正方形 anchor 适合小目标
        ),
        loss_cls=dict(type='FocalLoss', gamma=2.0, alpha=0.25),
        loss_bbox=dict(type='GIoULoss'),
    ),
)
```

5.3.3 使用 ultralytics 的 SimOTA (YOLOv8)

```
# 只需配置文件，SimOTA 是 YOLOv8 默认的标签分配策略
model: yolov8n.pt
data:IRSTD.yaml
epochs: 300
loss:
  cls_loss: FocalLoss
  box_loss: CIOULoss
```

5.4 OHEM 实现

```
class OHEMLoss(nn.Module):
    def __init__(self, ratio=0.25, loss_func=None):
        super().__init__()
        self.ratio = ratio          # 保留最难的 25% 样本
        self.loss_func = loss_func or F.cross_entropy

    def forward(self, logits, targets):
        losses = self.loss_func(logits, targets, reduction='none')
        sorted_losses, _ = torch.sort(losses, descending=True)
        keep = max(1, int(len(sorted_losses) * self.ratio))
        return sorted_losses[:keep].mean()
```

5.5 分层学习率实现

```
# 对不同部分使用不同学习率
optimizer = torch.optim.AdamW([
    {'params': model.backbone.parameters(), 'lr': 1e-5},      # backbone 小学习率
    {'params': model.neck.parameters(), 'lr': 5e-4},          # neck 中学习率
    {'params': model.head.parameters(), 'lr': 1e-3},           # 检测头大学习率
], weight_decay=1e-4)
```

5.6 梯度累积实现

```
accumulation_steps = 4    # 等效 batch_size = 4 × actual_batch_size

optimizer.zero_grad()
for i, (images, targets) in enumerate(train_loader):
    loss = model(images, targets) / accumulation_steps
    loss.backward()

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

5.7 Temperature Scaling 校准实现

```
class TemperatureScaling(nn.Module):
    def __init__(self):
        super().__init__()
        self.temperature = nn.Parameter(torch.ones(1))

    def forward(self, logits):
        return logits / self.temperature

# 在验证集上学习最优温度（不训练模型，只训练 T）
def calibrate(model, val_loader, device, epochs=50):
    model.eval()
    temp_scaling = TemperatureScaling().to(device)
    optimizer = torch.optim.LBFGS([temp_scaling.temperature], lr=0.01, max_iter=50)

    for _ in range(epochs):
        def closure():
            optimizer.zero_grad()
            loss = 0
            for images, targets in val_loader:
                logits = model(images.to(device))
                scaled = temp_scaling(logits)
                loss += F.cross_entropy(scaled, targets.to(device))
            loss /= len(val_loader)
            loss.backward()
            return loss
        optimizer.step(closure)

    print(f"Optimal temperature: {temp_scaling.temperature.item():.3f}")
    return temp_scaling
```

5.8 实现方式汇总

解决方案	是否需要手写	依赖库	放置位置
Weighted CE	不需要	<code>torch.nn.CrossEntropyLoss(weight=...)</code>	训练循环
Focal Loss	需要写 (~20行)	无额外依赖	训练循环

解决方案	是否需要手写	依赖库	放置位置
Dice Loss	需要写 (~10行)	无额外依赖	训练循环
GHM	需要写 (~30行)	无额外依赖	训练循环
OHEM	需要写 (~15行)	无额外依赖	训练循环
SE-Block	需要写或用 timm	timm 有内置	网络模型定义
CBAM	需要写 (~50行)	无	网络模型定义
ECA-Net	需要写 (~15行)	timm 有内置	网络模型定义
ATSS	不需要	mmdetection	配置文件
SimOTA	不需要	mmdetection / ultralytics	配置文件
OTA	不需要	mmdetection	配置文件
TOOD	不需要	mmdetection	配置文件
基础数据增强	不需要	torchvision.transforms	Dataset.getitem
Mixup/CutMix	不需要	timm.data.mixup	训练循环
Mosaic/Copy-Paste	不需要	ultralytics	配置文件
分层学习率	不需要	torch.optim 参数组	优化器定义
梯度累积	不需要	PyTorch 原生	训练循环
Temperature Scaling	需要写 (~20行)	无额外依赖	训练后校准
阈值搜索	需要写 (~10行)	sklearn / 手写	验证阶段

6. 各方法对比总结

6.1 方法适用矩阵

方法	轻度不平衡	中度不平衡	极端不平衡	实现难度	训练开销
Weighted CE	✓	○	X	★	低
Focal Loss	✓	✓	○	★★	低
OHEM	✓	✓	○	★★	中
GHM	✓	✓	○	★★★★	中
过采样	✓	○	X	★	低
SMOTE	✓	✓	○	★★	低
欠采样	✓	○	X	★	低
混合采样	✓	✓	✓	★★	中
Copy-Paste 增强	✓	✓	✓	★★	中
课程学习	○	✓	✓	★★★★	高
动态标签分配	✓	✓	✓	★★★★	中
阈值优化	✓	✓	✓	★	无
模型集成	✓	✓	✓	★★	高

✓ 推荐 ○ 可用但非最优 X 不推荐 ★ 简单 ★★★★★ 复杂

6.2 端到端组合方案

方案 A：通用分类任务（中度不平衡）

数据：SMOTE 过采样 + 轻度增强
网络：ResNet + SE 注意力
损失：Focal Loss ($\gamma=2, \alpha=0.25$)
训练：Class-Balanced Batch + 梯度累积
评估：PR-AUC + F1
部署：Temperature Scaling + 阈值搜索

方案 B：目标检测任务（重度不均衡）

数据：Mosaic + Copy-Paste 增强（少数类）

网络：FPN + 解耦头 + Anchor-Free 或动态分配（SimOTA）

损失：Focal Loss（分类）+ GIoU Loss（回归）

训练：OHEM + 两阶段训练

评估：mAP@0.5:0.95 + 各类 AP

部署：Soft-NMS + 类别级独立阈值

方案 C：极端不均衡（IR > 1000）

数据：GAN 生成少数类 + Copy-Paste + 重度增强

网络：强骨干 + 注意力 + 多尺度特征

损失：Focal Loss ($\gamma=5$) + Dice Loss

训练：课程学习 + 梯度累积 + 元学习微调

评估：PR-AUC + 少数类 Recall

部署：级联模型 + 规则辅助 + 增量学习反馈

7. 实践建议

7.1 推荐 workflow

- Step 1: 分析数据分布
 - 计算各类别数量、IR、分布可视化
- Step 2: 建立基线
 - 不做任何处理，用标准 CE 训练
 - 记录各类 Precision / Recall / F1
- Step 3: 逐步叠加策略（每次只加一种）
 - ① 先改损失函数（Focal Loss）
 - ② 再调采样策略
 - ③ 最后动网络结构
- Step 4: 验证集调参
 - 搜索最佳阈值
 - 校准模型输出概率
- Step 5: 部署监控
 - 监控各类别的线上表现
 - 建立反馈闭环

7.2 常见陷阱

陷阱	说明	正确做法
盲目追求 1:1	完全平衡不一定最优	保持轻微不均衡（1:3~1:5）通常更好
只看 Accuracy	被多数类掩盖	关注 PR-AUC 和少数类指标
过度过采样	少数类过拟合	配合增强或使用 SMOTE 而非简单复制
忽略验证集分布	验证集也做过采样	验证集必须保持真实分布
只训练不调阈值	默认 0.5 几乎总是差的	一定要做阈值搜索
过多技巧叠加	同时加所有方法反而更差	逐个实验，选择最有效的组合

7.3 关键文献索引

论文	会议	核心贡献
Focal Loss (Lin et al.)	ICCV 2017	聚焦难样本的损失函数
GHM (Li et al.)	AAAI 2019	梯度均衡化机制
ATSS (Zhang et al.)	CVPR 2020	自适应标签分配
OTA (Ge et al.)	CVPR 2021	最优传输标签分配
YOLOX (Ge et al.)	2021	SimOTA 动态分配
Class-Balanced Loss (Cui et al.)	CVPR 2019	有效样本数加权
SMOTE (Chawla et al.)	JAIR 2002	合成少数类样本
LDAM Loss (Cao et al.)	NeurIPS 2019	标签分布感知边距
BalanceCascade (Liu et al.)	ICDM 2009	级联式欠采样集成

报告生成日期: 2026-05-07